



Universidad de Granada

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y
MATEMÁTICAS

METAHEURÍSTICAS

Práctica 2:
Técnicas de Búsqueda de Poblaciones para el Problema
de Asignación Con Restricciones (PAR)

Autor: Jesús Muñoz Velasco
DNI: 76668409W
E-mail: jesusmuve18@correo.ugr.es
Horario prácticas: Martes 15:30 - 17:30 (Grupo A3)

Curso 2025-2026

Índice

1. Descripción del problema	2
2. Aplicación de los algoritmos empleados	3
2.1. Esquema de representación de soluciones	3
2.2. Esquema de representación de restricciones	3
2.2.1. Forma Matricial	3
2.2.2. Forma Vectorial	3
2.3. Criterio de validación de Soluciones	4
2.4. Reparación de soluciones no válidas	4
2.5. Evaluación de una solución (función objetivo)	4
2.5.1. Cálculo de centroides de una solución	4
2.5.2. Cálculo de la desviación de una solución	5
2.5.3. Cálculo de la infactibilidad de una solución	5
2.5.4. Evaluación de una solución	5
3. Estructura del método de búsqueda	6
3.1. Algoritmos Genéticos Generacionales con elitismo (AGG-UN y AGG-SF)	6
3.1.1. Operador de cruce uniforme (AGG-UN)	7
3.1.2. Operador de cruce por segmento fijo (AGG-SF)	7
3.1.3. Operador de mutación uniforme (AGG y AGE)	8
3.2. Algoritmos Genéticos Estacionarios (AGE-UN y AGE-SF)	8
3.3. Algoritmos Meméticos (AM-All, AM-Rand y AM-Best)	9
3.3.1. Operador de mutación por gen (AM)	9
3.3.2. Esquema principal de los AM	10
4. Estructura general del código	11
5. Experimentos y análisis de resultados	11
5.1. Descripción de casos del problema	11
5.2. Resultados obtenidos	12
5.2.1. Tablas globales por problema	13
5.2.2. Boxplots globales por problema	15
5.3. Análisis de resultados	16
5.3.1. Efecto del operador de cruce en los AGG	16
5.3.2. Efecto del operador de cruce en los AGE	16
5.3.3. Modelo generacional vs. modelo estacionario	17
5.3.4. Comparación entre Algoritmos Meméticos	17
5.3.5. Algoritmos Genéticos vs. Algoritmos Meméticos	18
5.3.6. Efecto del ratio de restricciones	18
5.3.7. Análisis de la variabilidad (boxplots)	18
5.3.8. Síntesis general	19

1. Descripción del problema

El Problema de Agrupamiento con Restricciones (PAR), también conocido como *Constrained Clustering*, consiste en la partición de un conjunto de n datos o instancias en k grupos (clústeres) disjuntos, basándose en la similitud de sus características y cumpliendo en la mayor medida posible un conjunto de restricciones impuestas sobre los pares de instancias.

Formalmente, sea $X = \{x_1, x_2, \dots, x_n\}$ un conjunto de n elementos en un espacio de dimensión d . El objetivo es encontrar una partición $C = \{C_1, C_2, \dots, C_k\}$ tal que se minimice la dispersión intra-clúster (medida como la distancia euclídea de cada elemento al centroide de su clúster), sujeta a un conjunto de restricciones a nivel de instancia (de tipo Must-Link y Cannot-Link).

Hay 2 tipos de restricciones posibles sobre 2 instancias $x_i, x_j \in X$:

-) *Must-Link* (ML). Los elementos x_i y x_j deben pertenecer al mismo clúster.
-) *Cannot-Link* (CL). Los elementos x_i y x_j no pueden pertenecer al mismo clúster.

Dado que encontrar una solución factible que satisfaga todas las restricciones es un problema NP-completo en la mayoría de configuraciones prácticas, relajaremos el incumplimiento de estas condiciones, donde las violaciones se penalizan en la función objetivo. La función objetivo a minimizar se define como

$$fitness(solucion) = desviacion(solucion) + \lambda \cdot infeasibilidad(solucion)$$

donde establecemos $\lambda = \frac{max_{dist}(X)}{restricciones(R)}$ para asegurar que la infactibilidad y la desviación tienen la suficiente relevancia. Además, la desviación de una solución se define como

$$desviacion(C_s) = \frac{1}{k} \sum_{C_i \in C_s} \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \left\| \vec{x}_j - \frac{1}{|C_i|} \sum_{x_j \in C_i} \vec{X}_j \right\|$$

y la infactibilidad como el número de condiciones incumplidas de una solución.

2. Aplicación de los algoritmos empleados

Para la resolución del PAR mediante los algoritmos Aleatorio, Greedy y Búsqueda Local, es necesario establecer un marco de trabajo común que defina cómo se representan, evalúan y manipulan las soluciones.

2.1. Esquema de representación de soluciones

Una solución S se representa como un vector de tamaño n (número de instancias), donde el índice i representa la instancia x_i , y el valor almacenado en $S[i]$ indica el índice del clúster al que ha sido asignada.

$$S = [c_0, c_1, \dots, c_{n-1}] \quad \text{donde} \quad c_i \in \{0, 1, \dots, k-1\}$$

Esta representación garantiza que toda instancia pertenezca a un clúster y solo uno.

2.2. Esquema de representación de restricciones

Para representar el conjunto de restricciones R se pueden usar dos estrategias principalmente, según convenga para mayor eficiencia en las evaluaciones de una solución:

2.2.1. Forma Matricial

El conjunto R se representa como una matriz de dimensión $n \times n$ donde cada elemento R_{ij} representará la restricción existente entre la instancia x_i y la x_j . Podrá tomar los siguientes valores:

-) $R_{ij} = 1$: Indica una restricción *Must-Link*.
-) $R_{ij} = -1$: Indica una restricción *Cannot-Link*.
-) $R_{ij} = 0$: Indica que no existe ninguna restricción entre x_i y x_j .

Por la naturaleza de esta matriz será cuadrada, simétrica y los elementos de la diagonal principal no podrán valer -1 . Es muy útil para saber si 2 elementos tienen alguna restricción entre ellos.

2.2.2. Forma Vectorial

En este caso se ha creado un nuevo tipo de dato llamado “Restricción” y tendrá la siguiente estructura:

```
1 struct Restriccion:
2     i, j          // Índices de los puntos  $x_i$  y  $x_j$  que participan en la restricción
3     tipo          // 1 para Must-Link, -1 para Cannot-Link
```

Código Fuente 1: Estructura de una restricción vectorial

De esta forma, el conjunto de restricciones R será un vector de “Restricciones” según acabamos de describir. Aunque esta estructura sea peor para saber si 2 elementos tienen alguna restricción, es mucho mejor para recorrer las restricciones una por una para cálculos como la infactibilidad.

2.3. Criterio de validación de Soluciones

Dado que el problema establece que hay un número k de clústeres para la solución se considerará que una solución S es válida si y solo si aparecen todos los clústeres en la solución, es decir, si se verifica que

$$c_i \in S \quad \forall c_i \in \{0, 1, \dots, k-1\}$$

2.4. Reparación de soluciones no válidas

El proceso de reparación de una solución S con clústeres vacíos consistirá en seleccionar un elemento $c_0 \in S$ que aparezca al menos 2 veces en el vector (para no dejar dicho clúster vacío) y sustituir su valor por el índice del clúster vacío. Se repite este proceso hasta que no haya clústeres vacíos. Seguiría el siguiente esquema:

```

1  Función RepararSolucion(S):
2      // Inicializamos el vector n_puntos con el número de puntos de cada clúster
   // en la solución S
3      n_puntos = nPuntosClusteres(S)
4
5      Para cada clúster c desde 0 hasta k-1:
6          Si n_puntos[c] == 0:
7              arreglado = Falso
8              instancia = 1
9
10             // Buscar una instancia de un clúster con más de 1 elemento
11             Mientras arreglado == Falso y instancia ≤ n:
12                 cluster_actual = S[instancia]
13
14                 Si n_puntos[cluster_actual] > 1:
15                     // Reasignar al clúster vacío
16                     S[instancia] = c_vacio
17                     n_puntos[cluster_actual] = n_puntos[cluster_actual] - 1
18                     n_puntos[c_vacio] = n_puntos[c_vacio] + 1
19                     arreglado = Verdadero
20
21                 instancia++
22
23     return S

```

Código Fuente 2: Pseudocódigo de la reparación de una solución

2.5. Evaluación de una solución (función objetivo)

La evaluación de una solución requiere calcular los centroides de los clústeres a partir de las asignaciones actuales en S , calcular la desviación media de cada clúster y contabilizar las restricciones violadas consultando la matriz R . Esto se hará de la siguiente forma:

2.5.1. Cálculo de centroides de una solución

```

1  Funcion obtenerCentroides(S):
2      n_puntos = nPuntosClusteres(S)
3
4      // Recorro todos los vectores sumándolos a su cluster
5      Para cada instancia i de X:
6          cluster = S[i]
7          Para cada dimension d de la instancia i:
8              centroides[cluster].coordenada[d] += X[i].coordenada[d]
9

```

```

10
11 // Divido entre el número de cada cluster para calcular la media
12 Para cada cluster c desde 0 hasta k-1
13     Para cada dimension d del centroide centroides[c]:
14         centroides[c].coordenada[d] /= n_puntos[c]
15
16 return centroides

```

Código Fuente 3: Pseudocódigo del cálculo de los centroides de una solución

2.5.2. Cálculo de la desviación de una solución

```

1 Función Desviacion(S):
2     centroides = obtenerCentroides(S)
3     n_puntos = nPuntosClusteres(S)
4
5     // Sumamos las distancias
6     Para cada instancia i de X:
7         cluster = S[i]
8         distanciaic[cluster] += distancia(X[i], centroides[cluster])
9
10    desviacion = 0
11
12    // Sumamos la media de las distancias
13    Para cada clúster c desde 0 hasta k-1
14        Si n_puntos[c] > 0:
15            desviacion += (distanciaic[c]/n)
16
17    return (desviacion/k)

```

Código Fuente 4: Pseudocódigo del cálculo de la desviación de una solución

2.5.3. Cálculo de la infactibilidad de una solución

```

1 Función Infactibilidad(S):
2     inf = 0
3
4     Para cada restricción res en el conjunto de restricciones R
5         juntos = (S[res.i] == S[res.j])
6
7         Si res es de tipo ML y !juntos:
8             inf++ // Error: Deberían estar juntos (Must-Link)
9         Sino si res es de tipo CL y juntos:
10            inf++ // Error: No deberían estar juntos (Cannot-Link)
11
12    return inf

```

Código Fuente 5: Pseudocódigo del cálculo de la infactibilidad de una solución

2.5.4. Evaluación de una solución

Finalmente, usando las funciones previamente descritas, y previamente fijado el valor de lambda, la función objetivo se calculará de la siguiente forma

```

1 Evaluacion(S) {
2     C = Desviacion(S)
3     inf = Infactibilidad(S)
4
5     return (C + lambda * inf)
6 }

```

Código Fuente 6: Pseudocódigo del cálculo de la función objetivo

3. Estructura del método de búsqueda

A continuación se describe en pseudocódigo la estructura de cada uno de los algoritmos implementados, reflejando fielmente las decisiones de diseño adoptadas. Los operadores comunes (reparación, evaluación, centroides, infactibilidad) ya fueron descritos en la sección anterior.

3.1. Algoritmos Genéticos Generacionales con elitismo (AGG-UN y AGG-SF)

Los dos algoritmos generacionales comparten el mismo esquema de evolución con elitismo y únicamente difieren en el operador de cruce empleado. La generación se realiza par a par (a diferencia de una arquitectura por lotes) de modo que selección, cruce, mutación, reparación y evaluación ocurren dentro del mismo bucle para cada pareja de hijos.

```

1  Función EvolveOneGeneration(P, fitnesses, bestSol, bestFit,
2                                operadorCruce, maxEvals):
3
4      // 1. Registrar el mejor de la generación ACTUAL antes de generar hijos.
5      best_old_idx = índice del mínimo en fitnesses
6
7      // 2. Precalcular qué cromosomas mutarán (esperanza matemática).
8      muta_hijo = vector de M booleanos con entero(Pm*M) valores Verdadero
9      Barajar(muta_hijo)
10
11     nueva_P      = vector vacío de tamaño M
12     nuevas_fitness = vector vacío de tamaño M
13
14     // 3. Generación par a par
15     Para i desde 0 hasta M-1 con paso 2:
16
17         // 3.1 Selección: dos torneos independientes de tamaño 3
18         p1_idx = Torneo(fitnesses)
19         p2_idx = Torneo(fitnesses)
20         hijo1  = P[p1_idx]
21         hijo2  = P[p2_idx]
22
23         // 3.2 Cruce: las (Pc*M/2) primeras parejas cruzan
24         Si (i/2) < entero(Pc * M / 2):
25             hijo1, hijo2 = operadorCruce(P[p1_idx], P[p2_idx])
26
27         // 3.3 Mutación de hijo1 (si le corresponde)
28         Si muta_hijo[i]:
29             gen_idx = aleatorio(0, n-1)
30             nueva_etiq = aleatorio distinto de hijo1[gen_idx] en {0,...,k-1}
31             hijo1[gen_idx] = nueva_etiq
32
33         // 3.4 Reparación y evaluación de hijo1
34         RepararSolucion(hijo1)
35         nueva_P[i] = hijo1
36         nuevas_fitness[i] = Evaluacion(hijo1); evals += 1
37         Si nuevas_fitness[i] < bestFit:
38             bestFit = nuevas_fitness[i]; bestSol = hijo1
39
40     Si evals >= maxEvals: return
41
42     // 3.5 Mismo proceso para hijo2
43     Si i+1 < M:
44         Si muta_hijo[i+1]:
45             gen_idx = aleatorio(0, n-1)
46             nueva_etiq = aleatorio distinto de hijo2[gen_idx] en {0,...,k-1}
47             hijo2[gen_idx] = nueva_etiq
48
49     RepararSolucion(hijo2)

```

```

50     nueva_P[i+1] = hijo2
51     nuevas_fitness[i+1] = Evaluacion(hijo2); evals += 1
52     Si nuevas_fitness[i+1] < bestFit:
53         bestFit = nuevas_fitness[i+1]; bestSol = hijo2
54
55     Si evals >= maxEvals: return
56
57     // 4. Elitismo: si el mejor padre supera al mejor hijo,
58     // sustituye al peor hijo para no perder la mejor solución
59     best_new_idx = índice del mínimo en nuevas_fitness
60     worst_new_idx = índice del máximo en nuevas_fitness
61
62     Si fitnesses[best_old_idx] < nuevas_fitness[best_new_idx]:
63         nueva_P[worst_new_idx] = P[best_old_idx]
64         nuevas_fitness[worst_new_idx] = fitnesses[best_old_idx]
65
66     P = nueva_P
67     fitnesses = nuevas_fitness

```

Código Fuente 7: Pseudocódigo de una generación del AGG con elitismo

El bucle principal del AGG simplemente invoca `EvolveOneGeneration` de forma reiterada hasta agotar el presupuesto:

```

1  Función AGG(operadorCruce, semilla):
2      // Parámetros: M=50, Pc=0.8, Pm=0.1, maxEvals=100000
3      P, fitnesses = InicializarPoblacion(M) // M cromosomas aleatorios válidos
4      evaluar P; evals += M
5      bestSol, bestFit = mejorIndividuo(P, fitnesses)
6
7      Mientras evals < maxEvals:
8          EvolveOneGeneration(P, fitnesses, bestSol, bestFit,
9                              operadorCruce, maxEvals)
10
11     return bestSol, bestFit

```

Código Fuente 8: Pseudocódigo principal del AGG

3.1.1. Operador de cruce uniforme (AGG-UN)

Para cada posición i del cromosoma se genera un aleatorio en $[0, 1]$; si es $< 0,5$ el primer hijo hereda el gen del primer padre (y el segundo hijo del segundo), en caso contrario se intercambian.

```

1  Función CruceUniforme(P1, P2):
2      Para cada posición i desde 0 hasta n-1:
3          Si aleatorio() < 0.5:
4              H1[i] = P1[i]; H2[i] = P2[i]
5          Sino:
6              H1[i] = P2[i]; H2[i] = P1[i]
7      return H1, H2

```

Código Fuente 9: Operador de cruce uniforme

3.1.2. Operador de cruce por segmento fijo (AGG-SF)

Se fija un segmento contiguo circular de longitud FN (posición inicial aleatoria en $[0, n - 1]$, longitud aleatoria en $[1, n/3]$). Dentro del segmento cada hijo hereda de un padre portador distinto; fuera del segmento se aplica cruce uniforme.

```

1  Función CruceSegmentoFijo(P1, P2):
2      length = aleatorio(1, n/3) // longitud del segmento
3      start = aleatorio(0, n-1) // inicio del segmento
4

```



```

5 // Marcar las posiciones del segmento (recorrido circular módulo n)
6 inSegmento = vector de n booleanos inicializado a Falso
7 Para j desde 0 hasta length-1:
8     inSegmento[(start + j) mod n] = Verdadero
9
10 Para cada posición i desde 0 hasta n-1:
11     Si inSegmento[i]:
12         // Dentro del segmento: padre portador diferente para cada hijo
13         H1[i] = P1[i]; H2[i] = P2[i]
14     Sino:
15         // Fuera del segmento: cruce uniforme
16         Si aleatorio() < 0.5:
17             H1[i] = P1[i]; H2[i] = P2[i]
18         Sino:
19             H1[i] = P2[i]; H2[i] = P1[i]
20 return H1, H2

```

Código Fuente 10: Operador de cruce por segmento fijo

3.1.3. Operador de mutación uniforme (AGG y AGE)

En AGG y AGE la mutación tiene semántica por cromosoma: si al cromosoma le corresponde mutar (determinado externamente por `muta_hijo` en AGG, o por un $\text{aleatorio} < P_m$ en AGE), se elige aleatoriamente un gen y se le asigna un valor de clúster distinto al actual.

```

1 Función Mutar(sol):
2     // Seleccionar un gen aleatorio y cambiar su etiqueta
3     gen_idx = aleatorio(0, n-1)
4     nueva_etiq = aleatorio distinto de sol[gen_idx] en {0,...,k-1}
5     sol[gen_idx] = nueva_etiq

```

Código Fuente 11: Operador de mutación uniforme por cromosoma (AGG y AGE)

3.2. Algoritmos Genéticos Estacionarios (AGE-UN y AGE-SF)

En el modelo estacionario solo se generan dos hijos por iteración, que compiten directamente con los dos peores de la población actual. Los operadores de cruce y mutación son los mismos que en el AGG.

```

1 Función AGE(operadorCruce, semilla):
2     // Parámetros: M=50, Pc=1.0, Pm=0.1, maxEvals=100000
3     P, fitnesses = InicializarPoblacion(M)
4     evaluar P; evals += M
5     bestSol, bestFit = mejorIndividuo(P, fitnesses)
6
7     Mientras evals < maxEvals:
8
9         // 1. Selección: dos torneos independientes de tamaño 3
10        p1_idx = Torneo(fitnesses)
11        p2_idx = Torneo(fitnesses)
12        hijo1 = P[p1_idx]; hijo2 = P[p2_idx]
13
14        // 2. Cruce
15        Si aleatorio() < Pc:
16            hijo1, hijo2 = operadorCruce(P[p1_idx], P[p2_idx])
17
18        // 3. Mutación por cromosoma (un gen aleatorio si le toca)
19        Si aleatorio() < Pm: Mutar(hijo1)
20        Si aleatorio() < Pm: Mutar(hijo2)
21
22        // 4. Reparación

```

```

23     RepararSolucion(hijo1);  RepararSolucion(hijo2)
24
25     // 5. Evaluación
26     fit1 = Evaluacion(hijo1);  evals += 1
27     hijo2_evaluado = Falso
28     Si evals < maxEvals:
29         fit2 = Evaluacion(hijo2);  evals += 1
30         hijo2_evaluado = Verdadero
31
32     // 6. Reemplazo estacionario
33     worst1_idx = índice del máximo en fitnesses
34     worst2_idx = índice del segundo máximo en fitnesses
35
36     Si fit1 < fitnesses[worst1_idx]:
37         P[worst1_idx] = hijo1
38         fitnesses[worst1_idx] = fit1
39
40     Si hijo2_evaluado y fit2 < fitnesses[worst2_idx]:
41         P[worst2_idx] = hijo2
42         fitnesses[worst2_idx] = fit2
43
44     // 7. Actualizar mejor global
45     Si fit1 < bestFit:
46         bestFit = fit1;  bestSol = hijo1
47     Si hijo2_evaluado y fit2 < bestFit:
48         bestFit = fit2;  bestSol = hijo2
49
50     return bestSol, bestFit

```

Código Fuente 12: Pseudocódigo del AGE (modelo estacionario)

3.3. Algoritmos Meméticos (AM-All, AM-Rand y AM-Best)

Los tres algoritmos meméticos hibridan el AGG con una Búsqueda Local Suave (BLS). La BLS se aplica cada 10 generaciones sobre un subconjunto de cromosomas que varía según la variante. A diferencia del AGG, la mutación en los AM está sobreescrita con semántica por gen, tal como se describe a continuación.

3.3.1. Operador de mutación por gen (AM)

En los AM la función *Mutar* se sobreescibe: cada gen del cromosoma muta independientemente con probabilidad P_m/n , por lo que en una misma llamada pueden mutar entre 0 y n genes. La esperanza de mutaciones por cromosoma es P_m , igual que en AGG/AGE, pero varios genes pueden verse afectados.

```

1  Función Mutar(sol):                // versión AM: por gen, no por cromosoma
2      Para cada posición i desde 0 hasta n-1:
3          Si aleatorio() < (Pm / n):
4              nueva_etiq = aleatorio distinto de sol[i] en {0,...,k-1}
5              sol[i] = nueva_etiq

```

Código Fuente 13: Operador de mutación por gen (AM, sobreescibe la versión AGG/AGE)

Búsqueda Local Suave (BLS)

La BLS recorre las posiciones del cromosoma en orden aleatorio (RSI) y para cada posición prueba todos los valores de clúster posibles salvo el actual, aceptando el primero que mejore el fitness. Mantiene dos contadores de evaluaciones: uno local a la llamada (*bls_evals*) y otro global (*evals*), ambos actúan como condición de

parada. Si el presupuesto se agota dentro del bucle interno de una posición, dicha posición **no** se contabiliza como fallo.

```

1  Función BLS(S, bestfit, maxevals_local, maxevals_global,  $\epsilon$ ):
2      // maxevals_local : límite de evals para ESTA llamada a BLS.
3      // maxevals_global: límite global del algoritmo (compartido).
4      //  $\epsilon$  : número máximo de fallos permitidos ( $\epsilon = 0.1*n$ ).
5
6      RSI      = PermutaciónAleatoria(0,...,n-1)
7      fallos   = 0; i = 0
8      bls_evals = 0          // contador LOCAL de esta llamada
9      newsol   = S
10
11     Mientras fallos <  $\epsilon$  y i < n
12         y bls_evals < maxevals_local y evals < maxevals_global:
13
14         p      = RSI[i]
15         oldvalue = S[p]
16         no_more_evals = Falso
17
18         // Probar todos los valores de clúster
19         Para val desde 0 hasta k-1:
20             Si val == oldvalue: continuar // omitir el cluster actual
21
22             Si bls_evals >= maxevals_local o evals >= maxevals_global:
23                 no_more_evals = Verdadero
24                 salir del bucle interno
25
26             newsol[p] = val
27             fit = Evaluacion(newsol)
28             bls_evals += 1; evals += 1 // descuenta del contador global
29
30             Si fit < bestfit:
31                 S      = newsol
32                 bestfit = fit
33             Sino:
34                 newsol[p] = S[p] // deshacer cambio en newsol
35
36             // Contar fallo solo si no hubo mejora Y el presupuesto no se agotó
37             // a mitad de exploración (si se agotó, no es un fallo real)
38             Si no no_more_evals y S[p] == oldvalue:
39                 fallos += 1
40
41         i += 1
42
43     return S, bestfit, bls_evals

```

Código Fuente 14: Pseudocódigo de la Búsqueda Local Suave (BLS)

3.3.2. Esquema principal de los AM

Las tres variantes difieren únicamente en qué cromosomas reciben la BLS cada 10 generaciones. El paso evolutivo es el del AGG (con la mutación por gen descrita arriba) y las evaluaciones de la BLS se descuentan del presupuesto global.

```

1  Función AM(variante, semilla):
2      // Parámetros: M=50, Pc=0.8, Pm=0.1, maxEvals=100000
3      // BLS: maxevals_local=100,  $\epsilon$ =0.1*n
4
5      P, fitnesses = InicializarPoblacion(M)
6      evaluar P; evals += M
7      bestSol, bestFit = mejorIndividuo(P, fitnesses)
8      generacion = 0
9
10     Mientras evals < maxEvals:
11
12         // 1. Paso evolutivo: una generación del AGG
13         EvolveOneGeneration(P, fitnesses, bestSol, bestFit,

```

```

14         CruceSegmentoFijo, maxEvals)
15     generacion += 1
16
17     // 2. Paso memético: cada 10 generaciones, aplicar BLS
18     Si generacion mod 10 == 0 y evals < maxEvals:
19
20         subconjunto = SeleccionarCromosomas(P, fitnesses, variante)
21
22         Para cada indice idx en subconjunto:
23             BLS(P[idx], fitnesses[idx], 100, maxEvals, entero(0.1*n))
24             Si fitnesses[idx] < bestFit:
25                 bestFit = fitnesses[idx]; bestSol = P[idx]
26
27     return bestSol, bestFit
28
29
30 Función SeleccionarCromosomas(P, fitnesses, variante):
31     Si variante == AM-All:
32         return {0, 1, ..., M-1}           // todos los cromosomas
33
34     Si variante == AM-Rand:
35         return {i : aleatorio() < 0.1}     // cada uno con prob. P_LS=0.1
36
37     Si variante == AM-Best:
38         ordenar indices por fitnesses ascendente
39         return los primeros entero(0.1*M) indices // los mejores (mín. 1)

```

Código Fuente 15: Pseudocódigo principal de los Algoritmos Meméticos

4. Estructura general del código

El código está programado sobre la API proporcionada de C++. Los ficheros creados principales se encuentran en la carpeta `inc`. El programa principal se encuentra en el fichero `main.cpp` que se podrá ejecutar de cualquiera de las siguientes formas:

```

1 main
2 main <seed>
3 main <seed> <filename> <ratio> <k>

```

Además se ha creado un script para compilarlo y otro para ejecutarlo, que se deberá ejecutar desde la carpeta raíz del proyecto (la inmediatamente superior a la carpeta que contiene el código). Por tanto, para compilar y ejecutar el proyecto tendremos que ejecutar desde una terminal

```

1 ./compilar.sh
2 ./ejecutar.sh

```

5. Experimentos y análisis de resultados

5.1. Descripción de casos del problema

Esta implementación se ha utilizado para los siguientes casos:

1. Zoo: En donde hay que agrupar animales en función de 16 atributos. Tiene 7 clases ($k = 7$) y 101 instancias.
2. Glass: En donde hay que agrupar distintos vidrios en función de 9 atributos sobre sus componentes químicos. Tiene 7 clases ($k = 7$) y 214 instancias.

3. Bupa: En donde hay que agrupar personas en función de sus hábitos de consumo de alcohol, definido con 5 atributos. Tiene 16 clases ($k = 16$) y 345 instancias.

Más adelante se especificarán el resto de parámetros

5.2. Resultados obtenidos

Se ha ejecutado una batería de pruebas con un programa python (`metrics.py`), ejecutando cada algoritmo para cada *dataset* y *ratio* para 10 semillas distintas, en concreto las siguientes:

```
1 SEEDS = [12345, 67890, 13579, 24680, 98765, 43210, 10293, 84756, 97531, 8642]
```

Código Fuente 16: Semillas utilizadas para las pruebas

Tras ejecutar el programa se han extraído las siguientes métricas:

Algoritmo	Dataset	Ratio	Fitness		Distancia		Incumplimiento		Evals	Tiempo (ms)	
			Media	Std	Media	Std	Media	Std		Media	Std
AGG-UN	bupa	15	0.1957	0.0030	0.1621	0.0049	0.0222	0.0018	100000	1383.4	14.18
AGG-UN	bupa	30	0.2170	0.0026	0.1562	0.0042	0.0403	0.0018	100000	2182.1	21.56
AGG-UN	glass	15	0.4191	0.0091	0.2961	0.0099	0.0577	0.0047	100000	742.3	61.28
AGG-UN	glass	30	0.4887	0.0063	0.2855	0.0091	0.0953	0.0039	100000	1017.1	27.76
AGG-UN	zoo	15	1.1273	0.1041	0.9670	0.1338	0.0435	0.0099	100000	341.2	19.19
AGG-UN	zoo	30	1.2277	0.0478	0.8730	0.0747	0.0963	0.0088	100000	426.5	3.54
AGG-SF	bupa	15	0.1964	0.0045	0.1628	0.0040	0.0223	0.0016	100000	1382.4	33.68
AGG-SF	bupa	30	0.2196	0.0039	0.1578	0.0052	0.0410	0.0016	100000	2162.5	12.19
AGG-SF	glass	15	0.4209	0.0079	0.2958	0.0136	0.0586	0.0031	100000	738.4	67.58
AGG-SF	glass	30	0.4902	0.0125	0.2911	0.0161	0.0934	0.0029	100000	1004.7	5.98
AGG-SF	zoo	15	1.1115	0.0735	0.9512	0.0883	0.0435	0.0086	100000	334.2	14.50
AGG-SF	zoo	30	1.2454	0.0514	0.8841	0.0769	0.0981	0.0087	100000	419.5	2.84
AGE-UN	bupa	15	0.1928	0.0024	0.1604	0.0034	0.0215	0.0019	100000	1390.5	4.22
AGE-UN	bupa	30	0.2181	0.0047	0.1575	0.0056	0.0401	0.0015	100000	2222.0	101.47
AGE-UN	glass	15	0.4224	0.0063	0.2960	0.0147	0.0593	0.0052	100000	761.0	63.94
AGE-UN	glass	30	0.4899	0.0123	0.2931	0.0176	0.0923	0.0038	100000	1026.6	5.19
AGE-UN	zoo	15	1.0911	0.0590	0.9236	0.0842	0.0455	0.0104	100000	341.7	14.83
AGE-UN	zoo	30	1.2409	0.0501	0.8801	0.0825	0.0980	0.0121	100000	429.3	3.95
AGE-SF	bupa	15	0.1926	0.0034	0.1583	0.0036	0.0227	0.0014	100000	1383.4	7.20
AGE-SF	bupa	30	0.2140	0.0021	0.1536	0.0033	0.0400	0.0016	100000	2231.3	159.72
AGE-SF	glass	15	0.4243	0.0110	0.3018	0.0158	0.0574	0.0037	100000	748.1	48.05
AGE-SF	glass	30	0.4874	0.0035	0.2874	0.0078	0.0938	0.0032	100000	1019.4	6.55
AGE-SF	zoo	15	1.0790	0.0655	0.9154	0.0811	0.0444	0.0082	100000	344.9	15.72
AGE-SF	zoo	30	1.2114	0.0525	0.8349	0.0706	0.1022	0.0070	100000	428.5	3.78
AM-All	bupa	15	0.1989	0.0038	0.1689	0.0041	0.0198	0.0016	100000	1134.4	14.95
AM-All	bupa	30	0.2181	0.0044	0.1587	0.0043	0.0393	0.0007	100000	1925.1	22.09
AM-All	glass	15	0.4144	0.0104	0.2903	0.0166	0.0582	0.0038	100000	612.3	16.47
AM-All	glass	30	0.4837	0.0114	0.2855	0.0217	0.0929	0.0059	100000	890.2	5.16
AM-All	zoo	15	1.1088	0.0688	0.9690	0.1015	0.0380	0.0102	100000	282.5	14.78
AM-All	zoo	30	1.2289	0.0625	0.8604	0.0800	0.1001	0.0073	100000	362.6	4.40
AM-Rand	bupa	15	0.1783	0.0027	0.1496	0.0032	0.0190	0.0014	100000	1276.6	50.96
AM-Rand	bupa	30	0.2039	0.0028	0.1467	0.0030	0.0379	0.0015	100000	2056.2	18.99

Algoritmo	Dataset	Ratio	Fitness		Distancia		Incumplimiento		Evals	Tiempo (ms)	
			Media	Std	Media	Std	Media	Std		Media	Std
AM-Rand	glass	15	0.4192	0.0049	0.2935	0.0082	0.0589	0.0040	100000	694.3	9.38
AM-Rand	glass	30	0.4877	0.0097	0.2888	0.0181	0.0932	0.0048	100000	980.3	6.91
AM-Rand	zoo	15	1.1053	0.0664	0.9300	0.0886	0.0476	0.0144	100000	325.5	15.47
AM-Rand	zoo	30	1.2732	0.0801	0.9415	0.0950	0.0901	0.0068	100000	408.0	2.67
AM-Best	bupa	15	0.1775	0.0029	0.1473	0.0032	0.0200	0.0009	100000	1260.3	12.21
AM-Best	bupa	30	0.2013	0.0015	0.1451	0.0024	0.0372	0.0019	100000	2057.8	14.33
AM-Best	glass	15	0.4174	0.0050	0.2968	0.0072	0.0566	0.0034	100000	699.1	18.02
AM-Best	glass	30	0.4819	0.0053	0.2804	0.0077	0.0945	0.0022	100000	980.8	11.08
AM-Best	zoo	15	1.1376	0.0656	0.9978	0.0757	0.0380	0.0079	100000	322.6	13.79
AM-Best	zoo	30	1.2600	0.0649	0.9180	0.0969	0.0929	0.0124	100000	408.0	3.59

Tabla 1: Resumen de métricas de rendimiento por algoritmo, dataset y ratio.

5.2.1. Tablas globales por problema

Con el objetivo de estudiar mejor este resultado rellenamos también las siguientes tablas con la información extraída de la tabla anterior

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
AGG-UN	1.1273	0.9670	0.0435	100000.0	341.2
AGG-SF	1.1115	0.9512	0.0435	100000.0	334.2
AGE-UN	1.0911	0.9236	0.0455	100000.0	341.7
AGE-SF	1.0790	0.9154	0.0444	100000.0	344.9
AM-All	1.1088	0.9690	0.0380	100000.0	282.5
AM-Rand	1.1053	0.9300	0.0476	100000.0	325.5
AM-Best	1.1376	0.9978	0.0380	100000.0	322.6

Tabla 2: Resultados obtenidos para el dataset **Zoo** con 15 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
AGG-UN	1.2277	0.8730	0.0963	100000.0	426.5
AGG-SF	1.2454	0.8841	0.0981	100000.0	419.5
AGE-UN	1.2409	0.8801	0.0980	100000.0	429.3
AGE-SF	1.2114	0.8349	0.1022	100000.0	428.5
AM-All	1.2289	0.8604	0.1001	100000.0	362.6
AM-Rand	1.2732	0.9415	0.0901	100000.0	408.0
AM-Best	1.2600	0.9180	0.0929	100000.0	408.0

Tabla 3: Resultados obtenidos para el dataset **Zoo** con 30 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
AGG-UN	0.4191	0.2961	0.0577	100000.0	742.3
AGG-SF	0.4209	0.2958	0.0586	100000.0	738.4
AGE-UN	0.4224	0.2960	0.0593	100000.0	761.0
AGE-SF	0.4243	0.3018	0.0574	100000.0	748.1
AM-All	0.4144	0.2903	0.0582	100000.0	612.3
AM-Rand	0.4192	0.2935	0.0589	100000.0	694.3
AM-Best	0.4174	0.2968	0.0566	100000.0	699.1

Tabla 4: Resultados obtenidos para el dataset **Glass** con 15 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
AGG-UN	0.4887	0.2855	0.0953	100000.0	1017.1
AGG-SF	0.4902	0.2911	0.0934	100000.0	1004.7
AGE-UN	0.4899	0.2931	0.0923	100000.0	1026.6
AGE-SF	0.4874	0.2874	0.0938	100000.0	1019.4
AM-All	0.4837	0.2855	0.0929	100000.0	890.2
AM-Rand	0.4877	0.2888	0.0932	100000.0	980.3
AM-Best	0.4819	0.2804	0.0945	100000.0	980.8

Tabla 5: Resultados obtenidos para el dataset **Glass** con 30 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
AGG-UN	0.1957	0.1621	0.0222	100000.0	1383.4
AGG-SF	0.1964	0.1628	0.0223	100000.0	1382.4
AGE-UN	0.1928	0.1604	0.0215	100000.0	1390.5
AGE-SF	0.1926	0.1583	0.0227	100000.0	1383.4
AM-All	0.1989	0.1689	0.0198	100000.0	1134.4
AM-Rand	0.1783	0.1496	0.0190	100000.0	1276.6
AM-Best	0.1775	0.1473	0.0200	100000.0	1260.3

Tabla 6: Resultados obtenidos para el dataset **Bupa** con 15 % de restricciones en 10 ejecuciones independientes (k=16).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
AGG-UN	0.2170	0.1562	0.0403	100000.0	2182.1
AGG-SF	0.2196	0.1578	0.0410	100000.0	2162.5
AGE-UN	0.2181	0.1575	0.0401	100000.0	2222.0
AGE-SF	0.2140	0.1536	0.0400	100000.0	2231.3
AM-All	0.2181	0.1587	0.0393	100000.0	1925.1
AM-Rand	0.2039	0.1467	0.0379	100000.0	2056.2
AM-Best	0.2013	0.1451	0.0372	100000.0	2057.8

Tabla 7: Resultados obtenidos para el dataset **Bupa** con 30 % de restricciones en 10 ejecuciones independientes (k=16).

5.2.2. Boxplots globales por problema

Adicionalmente se han generado los siguientes gráficos (script `graphics.py`) para visualizar la calidad de cada algoritmo:

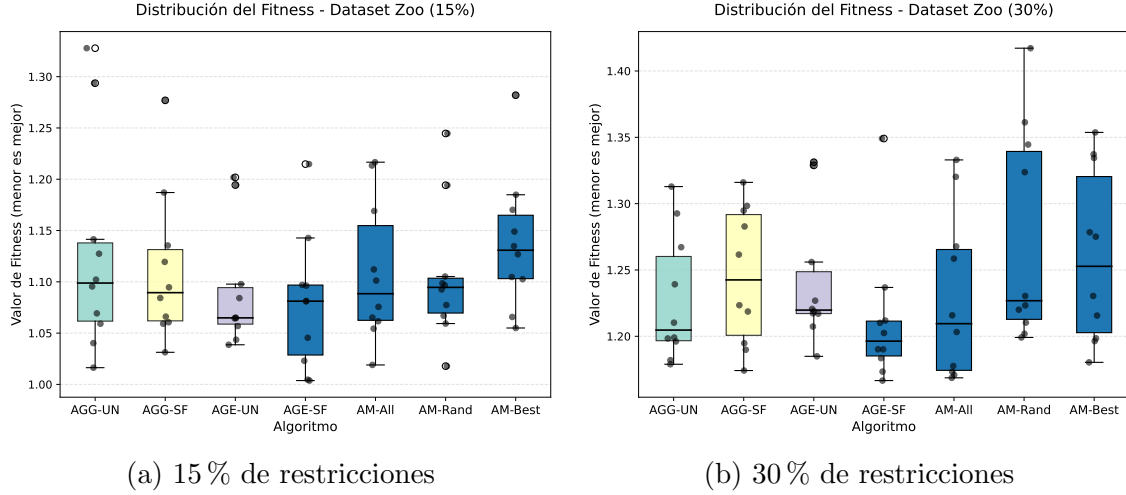


Figura 1: Distribución de los valores de Fitness obtenidos para el dataset **Zoo** en 10 ejecuciones independientes.

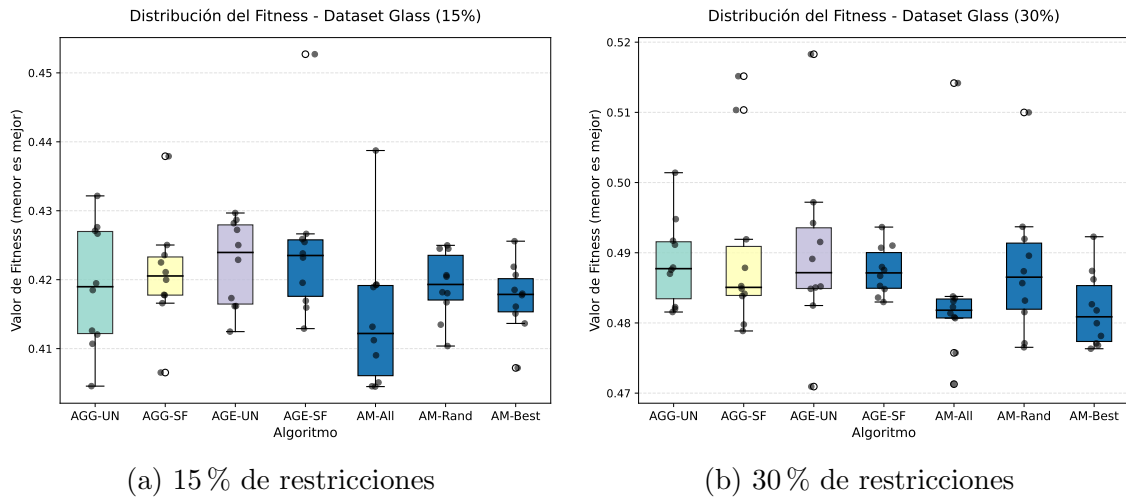


Figura 2: Distribución de los valores de Fitness obtenidos para el dataset **Glass** en 10 ejecuciones independientes.

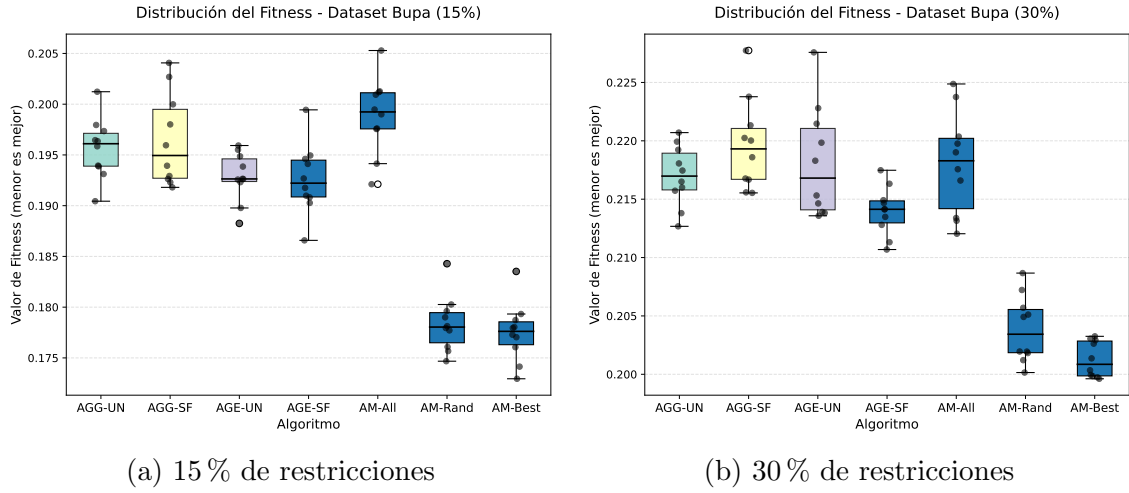


Figura 3: Distribución de los valores de Fitness obtenidos para el dataset **Bupa** en 10 ejecuciones independientes.

5.3. Análisis de resultados

El análisis se organiza comparando los algoritmos que comparten componentes, de modo que las diferencias observadas puedan atribuirse al elemento que los distingue.

5.3.1. Efecto del operador de cruce en los AGG

Comparando AGG-UN y AGG-SF en los tres datasets, se observa que las diferencias de *fitness* son pequeñas pero consistentes. En el dataset Zoo con 15 % de restricciones, AGG-SF obtiene un fitness medio de 1.1115 frente a 1.1273 de AGG-UN, lo que supone una mejora del 1,4 %. Sin embargo, con el 30 % de restricciones la situación se invierte ligeramente (1.2454 frente a 1.2277), lo que sugiere que el cruce uniforme preserve mejor la diversidad cuando el espacio de búsqueda está más limitado. En **Glass** y **Bupa**, las diferencias entre ambos cruces son prácticamente despreciables (inferiores al 0,5 %), lo que indica que para estos datasets ninguno de los dos operadores tiene una ventaja estructural clara en el esquema generacional.

Desde el punto de vista de los boxplots (Figuras 1, 2 y 3), tanto AGG-UN como AGG-SF presentan dispersiones similares, lo que confirma que la diferencia entre ambos operadores en el esquema generacional no es determinante.

5.3.2. Efecto del operador de cruce en los AGE

En el esquema estacionario, el cruce por segmento fijo (AGE-SF) supera sistemáticamente al cruce uniforme (AGE-UN) en todos los datasets y ratios excepto en Zoo con 15 %, donde las diferencias son mínimas (1.0790 frente a 1.0911). En Bupa con 30 % la ventaja es más notable: AGE-SF alcanza 0.2140 de fitness frente a 0.2181 de AGE-UN ($\approx 1,9$ % de mejora). Esto puede explicarse por el sesgo del cruce por segmento fijo hacia el padre *portador* del segmento: al utilizar siempre al mejor padre como portador para el primer hijo, se favorece la explotación de las mejores

soluciones, comportamiento más alineado con la selección competitiva del esquema estacionario.

5.3.3. Modelo generacional vs. modelo estacionario

Comparando los mejores representantes de cada esquema (AGG-UN y AGE-SF), el modelo estacionario ofrece soluciones de mayor calidad en la práctica totalidad de los casos. La diferencia es especialmente visible en Zoo: con el 15 % de restricciones, AGE-SF logra 1.0790 frente a 1.1273 de AGG-UN (4,3 % de mejora); con el 30 %, 1.2114 frente a 1.2277 (1,3 %). En Glass las ventajas son más modestas ($\approx 0,8\%$), y en Bupa el estacionario también es consistentemente mejor.

Este comportamiento es coherente con la teoría: el modelo estacionario aplica presión selectiva de forma más intensa porque cada par de hijos compite directamente con los peores individuos de la población, lo que acelera la explotación de zonas prometedoras del espacio de búsqueda. El modelo generacional, en cambio, reemplaza toda la población cada generación, lo que preserva más diversidad aunque puede ralentizar la convergencia hacia buenas soluciones.

En cuanto al tiempo de ejecución, ambos modelos son comparables dado que el criterio de parada es el número de evaluaciones (100000 en todos los casos).

5.3.4. Comparación entre Algoritmos Meméticos

Las tres variantes meméticas difieren en qué cromosomas reciben la Búsqueda Local Suave (BLS) cada 10 generaciones. Los resultados revelan patrones claros:

-) **AM-Best** es el mejor algoritmo memético en Bupa (el dataset de mayor tamaño con $k = 16$): obtiene 0.1775 con el 15 % de restricciones y 0.2013 con el 30 %, siendo el mejor resultado global en ambos casos. Al concentrar la BLS en los 5 mejores cromosomas de la población, se realiza una explotación intensa pero selectiva, lo que evita desperdiciar evaluaciones en individuos de baja calidad.
-) **AM-Rand** ofrece resultados muy competitivos en Bupa y es el mejor memético en algunos escenarios de ese dataset (15 % de restricciones: 0.1783). La aplicación probabilística de la BLS sobre un subconjunto variable introduce cierta variabilidad que puede ser beneficiosa cuando la población tiene buena diversidad.
-) **AM-All** es el algoritmo memético más débil en todos los datasets. Aplicar la BLS sobre todos los cromosomas consume muchas evaluaciones en individuos de baja calidad, reduciendo el número efectivo de generaciones evolutivas y, por tanto, la exploración global. En Bupa con 15 %, AM-All obtiene 0.1989, peor incluso que varios algoritmos puramente genéticos (AGE-UN: 0.1928, AGE-SF: 0.1926). Este efecto es menos pronunciado en Glass (0.4144 frente a 0.4191 de AGG-UN), lo que sugiere que cuando el problema tiene menos clústeres ($k = 7$), la BLS total no penaliza tanto.

En Zoo ninguno de los meméticos destaca claramente sobre los genéticos puros: AM-Rand obtiene 1.1053 con el 15 % (entre AGE-SF=1.0790 y AGG-UN=1.1273), lo que indica que en datasets pequeños ($n = 101$) el espacio de búsqueda es más manejable para los genéticos y la BLS no aporta tanto beneficio adicional.

5.3.5. Algoritmos Genéticos vs. Algoritmos Meméticos

La comparación global entre los mejores representantes de cada familia revela que los meméticos superan a los genéticos en los datasets más complejos. En **Bupa** ($n = 345$, $k = 16$), AM-Best y AM-Rand mejoran al mejor genético (AGE-SF: 0.1926) en un 7,8 % y 7,4 % respectivamente para el 15 % de restricciones. En **Glass** ($n = 214$, $k = 7$), la ventaja existe pero es más modesta (AM-All: 0.4144 frente a AGG-UN: 0.4191, mejora del 1,1 %).

En Zoo, sorprendentemente, los meméticos no superan de forma clara a los genéticos: el mejor resultado global lo obtiene AGE-SF (1.0790 con 15 %), mientras que AM-Best obtiene 1.1376, peor que varios algoritmos genéticos. Este comportamiento puede deberse al pequeño tamaño del dataset ($n = 101$): con pocas instancias y sólo 7 clústeres, el espacio de búsqueda es reducido y la BLS puede inducir convergencia prematura, perdiendo diversidad que el genético puro habría aprovechado.

5.3.6. Efecto del ratio de restricciones

Al aumentar el porcentaje de restricciones del 15 % al 30 %, todos los algoritmos ven incrementado su fitness medio, lo cual es esperable dado que el término de infactibilidad $\lambda \cdot \text{infeasibility}$ crece con el número de restricciones presentes. Sin embargo, el incremento relativo varía entre datasets: en Zoo el fitness aumenta aproximadamente un 13 % para la mayoría de algoritmos, en Glass en torno al 17 % y en Bupa alrededor del 11 %.

Más relevante es observar que la distancia intra-clúster disminuye al pasar del 15 % al 30 % en casi todos los casos. Por ejemplo, en AGE-SF para Zoo, la distancia pasa de 0.9154 (15 %) a 0.8349 (30 %). Esto indica que un mayor número de restricciones guía mejor al algoritmo hacia agrupamientos más compactos, aunque a costa de un mayor número de restricciones incumplidas, que prácticamente se duplica. En otras palabras, el aumento del ratio obliga al algoritmo a priorizar la reducción de distancias intra-clúster frente al cumplimiento estricto de todas las restricciones.

5.3.7. Análisis de la variabilidad (boxplots)

Los boxplots de las Figuras 1 y 3 revelan información adicional sobre la robustez de cada algoritmo:

- En **Zoo**, los algoritmos genéticos (especialmente AGE-SF y AGE-UN) presentan cajas más estrechas con algunos valores atípicos superiores, lo que indica que en la mayoría de ejecuciones convergen a soluciones de calidad similar, pero ocasionalmente quedan atrapados en óptimos locales pobres. Los meméticos AM-Rand y AM-Best presentan mayor dispersión, reflejando la variabilidad introducida por la selección aleatoria de individuos a mejorar.

-) En **Glass**, la dispersión es en general reducida para todos los algoritmos, siendo AM-All el que presenta la caja más baja y estrecha con el 15 % de restricciones, reflejando consistencia en la calidad obtenida.
-) En **Bupa**, AM-Best y AM-Rand muestran las distribuciones más bajas y compactas, especialmente con el 30 % de restricciones, lo que confirma que son los algoritmos más robustos y consistentes en este dataset.

5.3.8. Síntesis general

En conjunto, pueden extraerse las siguientes conclusiones:

1. El **esquema estacionario** supera al generacional en la mayoría de escenarios, especialmente cuando se combina con el cruce por segmento fijo (AGE-SF). La presión selectiva más intensa del modelo estacionario resulta ventajosa en todos los datasets evaluados.
2. El **cruce por segmento fijo** es ligeramente superior al uniforme en el esquema estacionario, mientras que en el generacional las diferencias son mínimas o inexistentes.
3. Los **algoritmos meméticos** mejoran significativamente a los genéticos en los problemas de mayor complejidad (Bupa, Glass), pero no siempre en problemas pequeños (Zoo). La variante **AM-Best** es la más recomendable cuando el problema es grande y complejo; AM-Rand ofrece un buen equilibrio entre explotación y diversidad.
4. **AM-All** es sistemáticamente el peor memético, ya que desperdiciar evaluaciones en BLS sobre cromosomas de baja calidad compromete la exploración global del espacio de búsqueda.
5. Un mayor **ratio** de restricciones conduce a soluciones con menor distancia intra-clúster pero más restricciones violadas, lo que refleja el compromiso inherente de la función objetivo utilizada.